

**SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL – SENAI
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS**

**MAYCON SIQUEIRA DE MORAES
PEDRO HENRIQUE MOURA DA SILVA**

**DOCUMENTAÇÃO TÉCNICA - VISCOM
SISTEMA DE VISÃO COMPUTACIONAL**

**NOVA LIMA – MG
2026**

**MAYCON SIQUEIRA DE MORAES
PEDRO HENRIQUE MOURA DA SILVA**

**DOCUMENTAÇÃO TÉCNICA - VISCOM
SISTEMA DE VISÃO COMPUTACIONAL**

Projeto de pesquisa e documentação técnica apresentado ao Curso Técnico de Desenvolvimento de Sistemas do SENAI, como requisito para o desenvolvimento do Sistema de Visão Computacional (VISCOM).

Instrutor: Frederico Martins Aguiar

**NOVA LIMA – MG
2026**

Resumo

Este documento apresenta a documentação técnica do VISCOM (Sistema de Visão Computacional), o componente central de controle da interface de análise visual de combustível integrado ao sistema Vector Zero-E-Mission. Desenvolvida em linguagem C# com a tecnologia WPF (Windows Presentation Foundation), a arquitetura do sistema adota o padrão MVVM (Model-View-ViewModel) por meio da classe principal **ViscomViewModel**. O ecossistema de software integra a biblioteca OpenCvSharp para captura e processamento de vídeo em tempo real, o framework MahApps.Metro para gerenciamento de interface gráfica e a biblioteca CsvHelper para exportação de dados em formato CSV. O sistema operacionaliza fluxos automatizados de captura de imagem, extração de métricas de cor (matiz, saturação e brilho) e turbidez, classificação de qualidade de amostras por regras técnicas, persistência assíncrona de resultados em banco de dados e calibração de novos padrões de combustíveis pelo operador. Pautado nos princípios do SOLID, injeção de dependência e programação assíncrona, o projeto demonstra alta resiliência e estabilidade, eliminando o congelamento da interface e garantindo o completo desacoplamento da lógica de negócios.

Sumário de Tabelas

Tabela 1 - Propriedades _____ 2

Sumário

1.	Introdução	1
2.	Arquitetura e Dependências	1
3.	Atributos e Propriedades	2
3.1.	Serviços Injetados (Princípio da Responsabilidade Única)	2
3.2.	Propriedades de Estado e Controle Gráfico	2
3.3.	Propriedades Computadas para Card de Resultado	3
4.	Comandos (ICommand)	3
5.	Detalhamento dos Fluxos e Métodos Principais	4
5.1.	Gerenciamento do Fluxo de Vídeo (Câmera)	4
5.2.	Fluxo de captura e Análise de Combustível (Analyze)	4
5.3.	Método de Salvamento no Banco (SalvarResultadoNoBancoAsync)	5
5.4.	Calibração do Sistema (CalibrateNew)	5
5.5.	Exportação de Dados (Save)	5
6.	Conclusão e Princípios de Engenharia Aplicados	6

1. Introdução

A classe **ViscomViewModel** é o componente central de controle da interface de análise visual de combustível (VISCOM) do sistema **Vector Zero-E-Mission**. Desenvolvida na linguagem **C#** utilizando a tecnologia **WPF (Windows Presentation Foundation)**, ela atua diretamente na camada **ViewModel** do padrão de arquitetura **MVVM (Model-View-ViewModel)**.

A principal responsabilidade desta classe é fazer a ponte entre a interface gráfica (XAML) e as regras de negócio de processamento de imagem, gerenciamento de câmera em tempo real, calibração de amostras e persistência de dados, garantindo que a interface permaneça responsiva e desacoplada da lógica interna.

2. Arquitetura e Dependências

A classe herda de **BaseViewModel** (que implementa a interface **INotifyPropertyChanged**), permitindo a atualização automática dos elementos da tela através de *Data Binding*.

As principais bibliotecas externas e namespaces utilizados são:

- **OpenCvSharp / OpenCvSharp.WpfExtensions:** Biblioteca de Visão Computacional utilizada para capturar os frames da câmera (VideoCapture), manipular as imagens em matrizes de pixels (Mat) e convertê-las para formatos compatíveis com o WPF (ToWriteableBitmap).
- **MahApps.Metro / MahApps.Metro.Controls.Dialogs:** Framework visual para criação de interfaces modernas e gerenciamento de caixas de diálogo assíncronas estilizadas.
- **CsvHelper:** Biblioteca utilizada para a exportação simplificada do histórico de análises para arquivos no formato CSV.
- **System.Windows.Threading.DispatcherTimer:** Timer utilizado para criar o laço de captura de vídeo a uma taxa de quadros controlada sem travar a interface do usuário.

3. Atributos e Propriedades

3.1. Serviços Injetados (Princípio da Responsabilidade Única)

A classe utiliza quatro serviços especializados para realizar suas tarefas, seguindo as boas práticas do SOLID:

- **_visionService:** Responsável por extrair as métricas visuais brutas da imagem.
- **_calibrationService:** Gerencia os padrões e limites de tolerância das amostras calibradas.
- **_classificationService:** Compara os dados da análise atual com a calibração para definir a qualidade do combustível.
- **_resultadosService:** Responsável por persistir os dados finais no banco de dados.

3.2. Propriedades de Estado e Controle Gráfico

Tabela 1 - Propriedades

Propriedade	Tipo	Descrição / Função
History	ObservableCollection <AnalysisRecord>	Coleção dinâmica que armazena o histórico das análises realizadas na sessão atual e atualiza a tabela na UI automaticamente.
CurrentResult	AnalysisResult?	Armazena o resultado da última análise executada. Dispara notificações para atualizar os cards informativos da tela.
CameraImageSource	BitmapSource?	Fornece o frame atual da câmera convertido em imagem para exibição do vídeo em tempo real no WPF.
FotoImageSource	BitmapSource?	Exibe a foto estática capturada que foi congelada para a realização da análise.
HabilitaStartCam / HabilitaStopCam	bool	Variáveis booleanas que controlam o estado de ativação/desativação dos botões da tela (evitando cliques duplicados).

3.3. Propriedades Computadas para Card de Resultado

Estas propriedades leem os dados do `CurrentResult` e formatam o texto/estilo diretamente para a tela:

- **ResultIcon:** Retorna um emoji representativo ( Sucesso,  Alerta,  Crítico,  Neutro) baseado no texto de qualidade.
- **ResultTitle:** Exibe o título principal do resultado da análise.
- **ResultDescription:** Monta uma string formatada contendo os valores de **H (Matiz)**, **S (Saturação)**, **V (Brilho)**, a **Turbidez**, se houve **Separação de Fases** e a data/hora exata.
- **ResultSeverity:** Retorna um enumerador (`ResultSeverity`) que permite ao XAML colorir dinamicamente o card de acordo com a gravidade do status (Verde, Amarelo, Vermelho ou Cinza).

4. Comandos (ICommand)

A interação do usuário com os botões da tela é mapeada através de comandos (`RelayCommand`), eliminando a necessidade de código direto no arquivo *code-behind* da View:

- **StartCamCommand:** Aciona o método `StartCam` para ligar a webcam.
- **StopCamCommand:** Aciona o método `StopCam` para desligar a webcam.
- **CapturaCommand:** Aciona o método `Captura` para congelar o frame atual da câmera.
- **AnalyzeCommand:** Aciona o método `Analyze` para processar a foto capturada.
- **CalibrateCommand:** Aciona o método `CalibrateNew` para cadastrar uma nova amostra padrão.
- **ClearHistoryCommand:** Aciona o método `ClearHistory` para limpar a tabela de histórico.
- **SaveCommand:** Aciona o método `Save` para exportar os dados salvos em memória para CSV.

5. Detalhamento dos Fluxos e Métodos Principais

5.1. Gerenciamento do Fluxo de Vídeo (Câmera)

- **StartCam():** Tenta abrir a câmera padrão do sistema (index 0) usando a API *DirectShow* (VideoCaptureAPIs.DSHOW). Se obtiver sucesso, inicia o `_timer` programado para rodar a cada **60 milissegundos** (gerando uma taxa de atualização de aproximadamente 16 frames por segundo).
- **UpdateFrame():** Método disparado a cada ciclo do timer. Ele faz a leitura do frame atual da câmera, clona o frame para uma matriz independente (`_currentFrame`) e atualiza a propriedade `CameraImageSource` para que o usuário veja a transmissão na tela.
- **StopCam():** Para o timer, libera os recursos de hardware da câmera (Release e Dispose), limpa as imagens da tela e redefine os estados dos botões de controle.

5.2. Fluxo de captura e Análise de Combustível (Analyze)

[Captura de Frame] -> [Extração de Métricas (HSV/Turbidez)] -> [Classificação por Regras] -> [Tratamento de Texto (Regex)] -> [Atualização da UI / Histórico] -> [Persistência Assíncrona no Banco]

O método `Analyze` executa de forma sequencial o seguinte algoritmo:

- **Validação:** Verifica se existe uma imagem estática capturada (`_capturedMat`).
- **Extração de dados:** Chama o `_visionService.Analyze` passando a imagem. Este serviço devolve as métricas de cor (H, S, V), turbidez e presença de separação de fases.
- **Classificação:** Envia o resultado parcial para o `_classificationService.Classify`, que define o status final da qualidade da amostra.
- **Sanitização com Expressão Regular:** Utiliza o método `Regex.Replace` para limpar textos indesejados ou repetitivos da string de qualidade (ex: removendo "– separação de fases detectada" caso necessário).
- **Atualização da UI:** Insere o registro no topo do histórico (`History.Insert(0, ...)`) e atualiza o `CurrentResult`.
- **Gravação no Banco de Dados:** Dispara de forma assíncrona o método `SalvarResultadoNoBancoAsync`.

5.3. Método de Salvamento no Banco (SalvarResultadoNoBancoAsync)

Este método cria um objeto do modelo ResultadoViscom, processando a inteligência de negócios complementar:

- Avalia os textos para definir qual ícone de status salvar.
- Gera observações automáticas com base nas faixas técnicas de turbidez (ex: define se o combustível está "visualmente limpo" caso a turbidez seja menor que 5, ou se há "possível contaminação ou oxidação" caso seja maior que 12).
- Envia o objeto para o `_resultadosService.SalvarResultadoViscomAsync(registro)` dentro de um bloco try-catch, garantindo que se houver falha de conexão com o banco, o sistema exiba um log de erro, mas não trave o aplicativo para o usuário.

5.4. Calibração do Sistema (CalibrateNew)

Permite ao operador treinar o sistema para reconhecer um novo tipo de combustível ou estado de contaminação:

1. Garante que uma foto foi tirada.
2. Abre uma caixa de texto assíncrona (`ShowInputAsync`) solicitando o nome da amostra (ex: s10_bom, agua).
3. Se o usuário preencher o nome, a foto atual é analisada pelo serviço de visão computacional.
4. As métricas resultantes são enviadas para o `_calibrationService.Save`, vinculando permanentemente aquele nome às métricas de cor e turbidez geradas.
5. Exibe uma mensagem de sucesso na tela mostrando detalhadamente as propriedades físicas calibradas.

5.5. Exportação de Dados (Save)

Utiliza a janela nativa do Windows (`SaveFileDialog`) configurada para filtrar extensões `.csv`. Caso o usuário selecione um caminho válido, a biblioteca `CsvHelper` abre um fluxo de escrita de texto codificado em **UTF-8** e descarrega toda a lista contida na propriedade `History` em um arquivo de planilha estruturado de forma automática.

6. Conclusão e Princípios de Engenharia Aplicados

A classe ViscomViewModel demonstra maturidade de desenvolvimento de software técnico ao implementar os seguintes conceitos:

- **Padrão MVVM:** Separação total de conceitos de UI e lógica de controle.
- **Programação Assíncrona (async/await):** Utilizada no salvamento do banco e na abertura de diálogos, impedindo o congelamento da interface gráfica (*UI Freeze*).
- **Injeção de Dependência/Inversão de Controle:** Utilização de serviços externos especializados e focados em suas respectivas tarefas.
- **Tratamento de Exceções Robustas:** Uso estruturado de blocos try-catch em operações críticas (câmera e banco de dados) garantindo a resiliência e estabilidade do software em ambiente de produção.